

After START comes the actual bits.

Example byte:

10110010

I²C sends the **Most Significant Bit (MSB) first**:

1
0
1
1
0
0
1
0

Important Rule

While transmitting data:

SDA changes only when SCL is LOW

When SCL becomes HIGH:

data must remain stable

The receiver reads the SDA value during the HIGH period of SCL.

Example

Suppose we want to send:

1

The sequence is:

SCL LOW

Set SDA = 1

SCL HIGH

Receiver reads 1

SCL LOW

Next bit

ACK Bit

After every 8 bits:

8 data bits

+

1 ACK bit

The receiver pulls SDA LOW to acknowledge.

ACK = SDA LOW

NACK = SDA HIGH

This is extremely important in debugging I²C systems.

Example Transaction

START

ADDRESS

ACK

DATA

ACK

DATA

ACK

STOP

This is the sequence you'll see with sensors like MPU6050, BMP280, OLEDs, EEPROMs, etc.

I²C Learning Notes (Single ESP32 Master ↔ Slave Loopback)

Objective

Learn the I²C protocol using only:

- 1 × ESP32
- 2 jumper wires
- ESP-IDF v6.1

No sensors, OLEDs, or external devices.

Hardware Setup

Initial master-only setup:

GPIO12 = SDA (Master)

GPIO14 = SCL (Master)

Later we created a slave on the same ESP32:

GPIO12 ---- GPIO27 SDA

GPIO14 ---- GPIO26 SCL

Final architecture:

ESP32

I2C_NUM_0

Master

GPIO12 SDA

GPIO14 SCL

|
| Jumpers
|

I2C_NUM_1

Slave

GPIO27 SDA

GPIO26 SCL

Address = 0x55

Theory Learned

Idle State

SDA = HIGH

SCL = HIGH

Pull-up resistors keep both lines HIGH.

START Condition

SCL HIGH
SDA HIGH → LOW

Meaning:

Communication begins

STOP Condition

SCL HIGH
SDA LOW → HIGH

Meaning:

Communication ends

Data Transfer Rule

SCL LOW → SDA may change
SCL HIGH → SDA must remain stable

Exceptions:

START
STOP

ACK/NACK

After every 8 bits:

8 Data Bits
+
1 ACK Bit

ACK:

SDA LOW

NACK:

SDA HIGH

Addressing Learned

Example:

Slave Address = 0x3C

Write:

0x78

Read:

0x79

Formula:

$(\text{Address} \ll 1) \mid \text{R/W}$

Practical Work

Step 1 — Hello World

Code:

```
#include <stdio.h>
```

```
void app_main(void)
```

```
{  
    printf("Hello, World!\n");  
}
```

Error 1

Problem

```
return 0;
```

inside:

```
void app_main()
```

Compiler Error

return with a value in function returning void

Fix

Remove:

```
return 0;
```

Step 2 — I²C Master Initialization

Successfully initialized:

```
I2C_NUM_0  
GPIO12 SDA  
GPIO14 SCL
```

Log:

```
I2C Master Initialized
```

Step 3 — First Transmission Attempt

Transmission:

```
i2c_master_transmit(  
    NULL,  
    &data,  
    1,  
    1000  
);
```

Error 2

Log:

i2c handle not initialized
Transmit Result = ESP_ERR_INVALID_ARG

Cause

Used:

NULL

instead of a device handle.

Fix

Create device:

i2c_master_bus_add_device()

and use:

dev_handle

Step 4 — Missing Brace Error

Compiler Error:

expected declaration or statement at end of input

Cause

Missing:

}

at end of:

app_main()

Fix

Add closing brace.

Step 5 — First Real NACK

Master created:

Bus

Device Address = 0x55

No slave existed.

Log:

I2C Master Initialized

Device 0x55 Added

Transmit Result = ESP_ERR_INVALID_RESPONSE

Meaning

Transaction:

START

ADDRESS 0x55

NACK

STOP

No slave responded.

Step 6 — Creating Slave

Created:

I2C_NUM_1

Address = 0x55

GPIO27 SDA

GPIO26 SCL

Error 3

Compiler:

implicit declaration of function xTaskCreate

Cause

Missing FreeRTOS headers.

Fix

Add:

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
```

Error 4

Linker Error:

undefined reference to i2c_slave_task

Cause

slave.c not compiled.

Current:

```
idf_component_register(
    SRCS "i2c_learning.c"
)
```

Fix

```
idf_component_register(
    SRCS
        "i2c_learning.c"
        "slave.c"
    INCLUDE_DIRS "."
)
```

Error 5

Slave Runtime Error

Log:

ringbuffer create failed

Slave Init = ESP_ERR_INVALID_STATE

Cause

Incomplete slave configuration.

Missing:

.receive_buf_depth

Fix

.send_buf_depth = 256,
.receive_buf_depth = 256,

Error 6

FreeRTOS Task Crash

Log:

FreeRTOS Task "i2c_slave" should not return

Cause

Task ended.

Fix

Keep task alive:

```
while(1)
{
    vTaskDelay(pdMS_TO_TICKS(1000));
}
```

First Successful Slave Initialization

Log:

I (299) SLAVE: Slave Init = ESP_OK

Meaning:

Slave created successfully

Address = 0x55

First Successful I²C Transaction

Log:

I (299) I2C: I2C Master Initialized

I (299) I2C: Device 0x55 Added

I (299) I2C: Transmit Result = ESP_OK

Meaning:

START

ADDRESS 0x55

ACK

DATA

ACK

STOP

The slave acknowledged.

I²C Scanner Project

Created scanner:

0x01 → 0x7F

For every address:

START

ADDRESS

Wait ACK

Scanner Result

Log:

```
I (300) SLAVE: Slave Init = ESP_OK  
I (300) I2C: I2C Master Initialized  
I (300) I2C: Starting I2C Scan...  
I (320) I2C: Found device at 0x55  
I (330) main_task: Returned from app_main()
```

What This Proved

The scanner successfully discovered:

Address = 0x55

Meaning:

Master

↓

START

↓

Address 0x55

↓

Slave ACK

↓

STOP

Communication exists.

Skills Learned

- ✓ SDA and SCL
- ✓ START condition
- ✓ STOP condition
- ✓ ACK
- ✓ NACK

- ✓ 7-bit addressing
 - ✓ Device handles
 - ✓ I²C master creation
 - ✓ I²C slave creation
 - ✓ ESP-IDF v6.1 I²C APIs
 - ✓ FreeRTOS tasks
 - ✓ CMake source registration
 - ✓ I²C scanner implementation
 - ✓ Master ↔ Slave communication on a single ESP32
-

Final Achievement

Built a complete I²C network using:

1 ESP32
2 Jumper Wires
ESP-IDF v6.1

with:

Master (I2C_NUM_0)
↕
Slave (I2C_NUM_1)
↕
Address Discovery
↕
Successful ACK
↕
Successful Transaction